

PETsys
Electronics

TOFPET2 ASIC Evaluation Kit

Software user guide v2019.09

Contents

1	System Overview	7
1.1	TOFPET2 ASIC Evaluation Kit (before February 2017)	7
1.1.1	Programming the ML605 FPGA board	7
1.1.2	Hardware switch and FMC cable connection	8
1.2	TOFPET2 ASIC Evaluation Kit (after February 2017)	8
1.2.1	Programming the FEB/D FPGA	9
1.3	Computer requirements	9
1.3.1	ROOT	10
1.4	Software	10
1.5	Before first use	11
2	Quickstart	13
2.1	Starting the GUI	14
2.1.1	Launching “daqd”	14
2.2	Configuration and calibration	14
2.3	Data Acquisition with a radioactive ^{22}Na point source	16
2.3.1	Coincidence events and Coincidence Time Resolution	16
3	Reference Configuration File and Calibration procedure	19
3.1	Configuration and calibration	21
3.1.1	Discriminator calibration	22
3.1.2	TDC calibration	22
3.1.3	QDC calibration	25
3.1.4	QDC charge measurement linearisation / Energy Calibration (Optional)	25
3.2	Map tables	26
3.2.1	Channel map table contents	26
3.2.2	Trigger map table contents	26
3.2.3	Creating a simple map	27
4	Data acquisition	29
4.1	Data acquisition template	29
4.2	Data acquisition procedure	30
4.2.1	Parameter scan	31
4.2.2	Hardware coincidence trigger	32
4.3	Temperature reading and stability	33
4.4	Other utility applications	33
4.5	Using an external trigger	33
5	Data processing	35
5.1	Singles application	35
5.2	Coincidence application	37
5.3	Group application	39
5.4	Raw to raw application	40
5.5	Data Processing with GUI	41

Release notes

Version 2019.09

Support for mixed mode (TOT and QDC) acquisition.

Version 2019.08

New energy calibration/linearisation introduced in software and documented. New version of “acquire_sipm_data”, allowing to acquire data while varying some key parameters.

Version 2019.01

New Graphical User Interface (GUI).

Version 2018.09

Updated software usage, new functions for added functionalities. New code example for multiple step data acquisition. New plot for QDC calibration summary.

Version 2018.07

Software revision, with new usage commands on several programs. Compatible with new BIAS-16P mezzanine. Requires new FEB/D firmware (v2018.06.29 or later).

Version 2018.05

Data acquisition with external gate signal.

Version 2018.04

Reference to temperature stabilization and usage of “convert_raw_to_group”

Version 2018.02

Major revision and new section regarding the usage of an external trigger.

Version 2017.11

Changed plots for calibration diagnosis and example CTR plot in Quickstart chapter to reflect new QDC calibration.

Version 2017.06

First version of Software package and Software User Guide for TOFPET2 ASIC.

Chapter 1

System Overview

For Software download, future updates (we recommend checking for updates every month or two), documentation and table maps, please check the PETsys Electronics Client Package area under the URL link: <https://drive.google.com/folderview?id=0Byh2q2A3bm1hQk5HZGROYWFURUk>
The following sections provide a brief overview of the hardware and software specifications.

1.1 TOFPET2 ASIC Evaluation Kit (before February 2017)

If the user upgraded from a TOFPETv1 evaluation kit shipped before February 2017, the system is composed of (as illustrated by figure 1.1):

- A Power Adapter Board (PAB), which can hold up to two TOFPET2 ASIC test boards.
- Two TOFPET2 ASIC test boards and respective adaptors (2 adapters needed per board)
- A Xilinx ML605 Virtex6 evaluation kit.
- A computer, running 64-bit Linux.

The two TOFPET2 ASIC test boards contain one ASIC each, SiPM connectors and some power supply and filtering.

The PAB provides the test boards with power, bias voltage for the SiPM, a filtered low jitter clock (200 MHz) and a FPGA interface.

The ML605 FPGA implements the communication logic necessary to configure the ASICs, the clock filter and the HV DAC which produces the bias voltages, as well as reading out the data produced by the ASIC. It can also generate a test pulse, with an adjustable delay in relation to the ASIC reference clock, which is delivered to the ASICs through the PAB interface but is also available, as a 2.5 V LVCMOS signal, on the GPIO_USER_P connector on the ML605 board. This can be used for various measurements, from triggering the TDCs inside the ASIC with known test pulse to triggering a laser driver.

An external test pulse can also be used, by providing a 2.5 V LVCMOS signal on the GPIO_USER_N connector on the ML605 board.

Communication between the FPGA and the computer is handled via Gigabit Ethernet.

1.1.1 Programming the ML605 FPGA board

Instructions to program the ML605 FPGA board can be found in <http://e-lab.github.io/html/wiki-program-ml605-poweron-flash.html> or http://www.xilinx.com/support/documentation/application_notes/xapp518-isp-bpi-prom-virtex-6-pcie.pdf. The firmware (.bit) files to be uploaded are provided by PETsys Electronics in the Client Package folder.

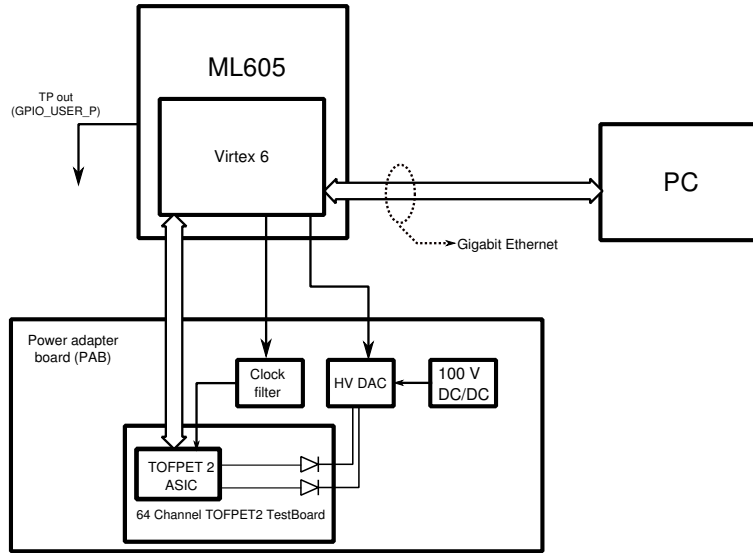


Figure 1.1: System Diagram - TOF ASIC Evaluation Kit

1.1.2 Hardware switch and FMC cable connection

The user must ensure that the ML605 SW1 switch bank has a proper configuration.

Position 1 and 2 configure clock enables and, for testing purposes, they allow the clock to the ASIC to be gated off. For normal operation, the configuration must be:

- Position 1 set to OFF
- Position 2 set to ON

Position 3 enables the external test pulse source. The GPIO_USER_N will see a logic HIGH when left floating. Unless a source of external test pulses is active and being used, position 3 of SW should be set to OFF.

Also, make sure that the FMC cable connecting to the PAB is placed on the LPC connector (J63) of the ML605 board.

1.2 TOFPET2 ASIC Evaluation Kit (after February 2017)

If the user is upgraded from a TOFPETv1 evaluation kit purchased after February 2017 or has recently purchased a new TOFPET2 Evaluation Kit, the system is composed of (as illustrated by figure 1.2):

- Front end boards - type D (FEB/D) set, each being able to hold up to 8 TOFPET2 ASIC test boards. The FEB/D have a HVDAC module and a Gigabit Ethernet mezzanine, allowing to connect to the computer with GBE.
- Two TOFPET2 ASIC test boards (or two FEM64) and cables.
- A computer, running 64-bit Linux.

The TOFPET2 ASIC test boards contain one ASIC each, SiPM connectors and some power supply and filtering. The FEM64 are composed of 1 front end board - type A (FEB/A, holding one TOFPET2 ASIC), 1 dummy FEB/A (without ASIC), 1 front end board - type I (FEB/I) and 1 front end board - type S (FEB/S).

The FEB/D set provide the TOFPET2 ASIC test boards with power, bias voltage for the SiPM, a filtered low jitter clock (200 MHz) and a FPGA interface, which implements the communication logic necessary to configure the ASICs, the clock filter and the HV DAC which produces the bias voltages, as well as reading out the data produced by the ASIC. It can also generate a test pulse, with an adjustable delay in relation to the ASIC reference clock. This can be used for various measurements, from triggering the TDCs inside the ASIC with known test pulse to triggering a laser driver.

Communication between the FEB/D FPGA and the computer is handled via Gigabit Ethernet.

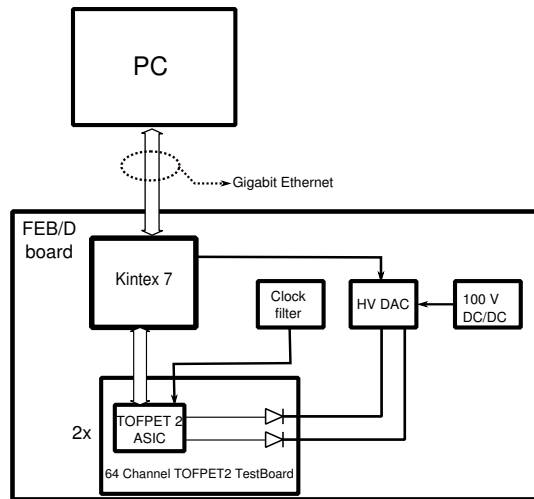


Figure 1.2: System Diagram - TOFPET2 ASIC Evaluation Kit

1.2.1 Programming the FEB/D FPGA

Instructions to program the FEB/D FPGA can be found in the manual “TOF FEB-D.v2 - Hardware User Guide (v1.0)”. The firmware (.bit) files to be uploaded are provided by PETsys Electronics in the Client Package folder.

1.3 Computer requirements

The data acquisition computer must be capable of running 64-bit Linux and must have one available Gigabit Ethernet port. The Gigabit Ethernet interface only works at 1 Gbit/s, it is not capable of communicating with slower 10 or 100 Mbit/s ports.

This Ethernet port needs to be configured with the following parameters.

- IP address: 192.168.1.1
- Netmask: 255.255.255.0

The operative system must be 64-bit Linux, preferably CentOS 7 or higher versions (<http://www.centos.org/>). If using CentOS 7, a script is provided with the Software to install all software dependencies and requirements.

For reference, the full list of requirements to compile PETsys Electronics software is described below:

- C++ development tools (GCC, Make, libstdc++, etc). If using CentOS 6.x or similar, choosing the “Software Development Workstation” profile should install most of them.
- cmake
- ROOT (<http://root.cern.ch/>) – tested with v5.34/09. It can also be installed from the EPEL repository (<https://fedoraproject.org/wiki/EPEL>).
- Python 2.6. If using CentOS 6.x or similar, it should come as part of the base installation.
- Python bitarray module (<https://pypi.python.org/pypi/bitarray/>) – tested with v0.8.1.
- Python CRC module (<https://pypi.python.org/pypi/crcmod/>) – tested with v1.7.
- Python serial module (<http://pyserial.sourceforge.net/pyserial.html>) – tested with v2.7.
- Python POSIX IPC module (https://pypi.python.org/pypi/posix_ipc/) – tested with 0.9.4

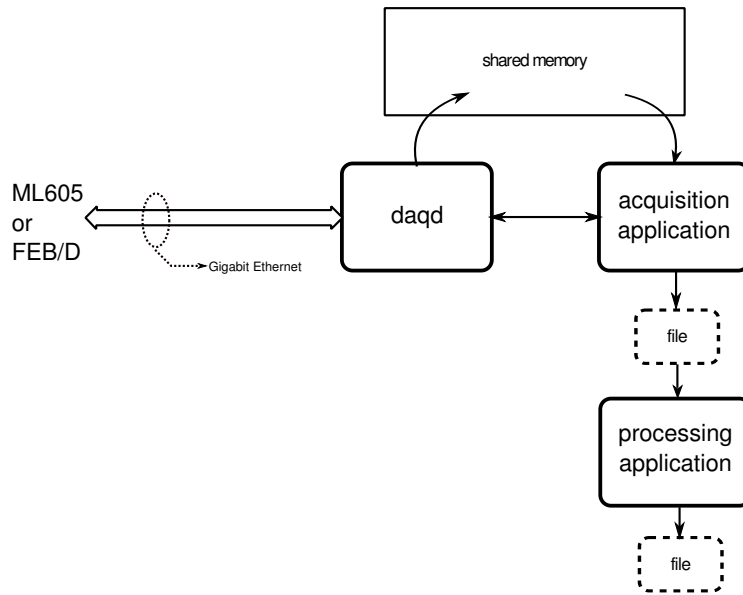


Figure 1.3: Software diagram

- Python numpy module (<https://pypi.python.org/pypi/numpy>) – tested with 1.4.1
- Python argparse module (<https://pypi.python.org/pypi/argparse>) – tested with 1.3.0

1.3.1 ROOT

If installing ROOT from packages, note that the following packages need to be installed.

- root-gui-fitpanel
- root-spectrum
- root-spectrum-painter
- root-minuit2
- root-python

1.4 Software

The computer is used to run a suite of software in order to configure the system and process the data produced by the ASICs.

As shown in figure 1.3, the acquisition software consists of three main parts.

“daqd” is a communication server which must always be running in order to communicate with the hardware. Its main purposes are to abstract away the communication method and to act as a buffer to the data provided by the hardware.

The acquisition applications, communicating with the hardware through “daqd”, configure the hardware and acquire the data into files as needed.

Finally, processing applications are used to process and analyse the acquired data.

Communication between “daqd” and the acquisition applications is done by a UNIX socket (“/tmp/d.sock”) and a POSIX shared memory block (“/dev/shm/daqd_shm”).

A Graphical user interface (GUI) can be used to configure and run all the applications, including “daqd” server.

1.5 Before first use

After downloading the software archive and copying the file to any desired folder, extract it by typing in a command line shell:

```
tar xvzf TOFPET2_SOFTWARE_vYYYY.MM.tar.gz
```

If using CENTOS 7, install all software requirements by typing:

```
su  
sh INSTALL_SW_REQUIREMENTS_CENTOS.sh  
exit
```

The next step is to compile the C++ applications. Navigate to the newly created base folder (“sw_daq_tofpet2”) of the decompressed software. Then execute the following commands:

```
cmake .  
make
```


Chapter 2

Quickstart

The purpose of this chapter is to guide the user to quickly acquire data with your recently acquired hardware, process such data searching for coincidence events and plot charge/energy spectra and coincidence time resolution (CTR). For a detailed description on each step please address to the next chapters. It is expected that the user has a point source isotope emitting 2 Gamma photons of 511 keV, typically a ^{22}Na point source with an activity higher than 10 μCi (370 kBq). The Gamma photons are detected by either a single or an array of SiPMs coupled to scintillation crystal(s) and connected to PETsys Electronics TOFPET2 ASIC Test boards or FEB/S via the SAMTEC connector(s). In order to measure events in coincidence, the ^{22}Na source should be placed in a virtual line between a pair (or several pairs in the case of two arrays) of crystal+SiPM pixels. If using a recent eKit version with a FEB/D, the FEM boards should be connected in ports F1 and F3 of the FEB/D.

The user should make sure that all hardware is properly connected, powered and the correct firmware is loaded in the FEB/D FPGA. The Ethernet cable should be plugged and the PC Ethernet card configured as described in Chapter 1.

The setup should be placed inside a dark box to avoid any light contamination and preferably with some sort of active cooling (a cooling system is provided for eKits purchased after February 2017). If no active cooling is available, the user should at least place a small fan to avoid heat concentration on the ASICs and SiPMs.¹

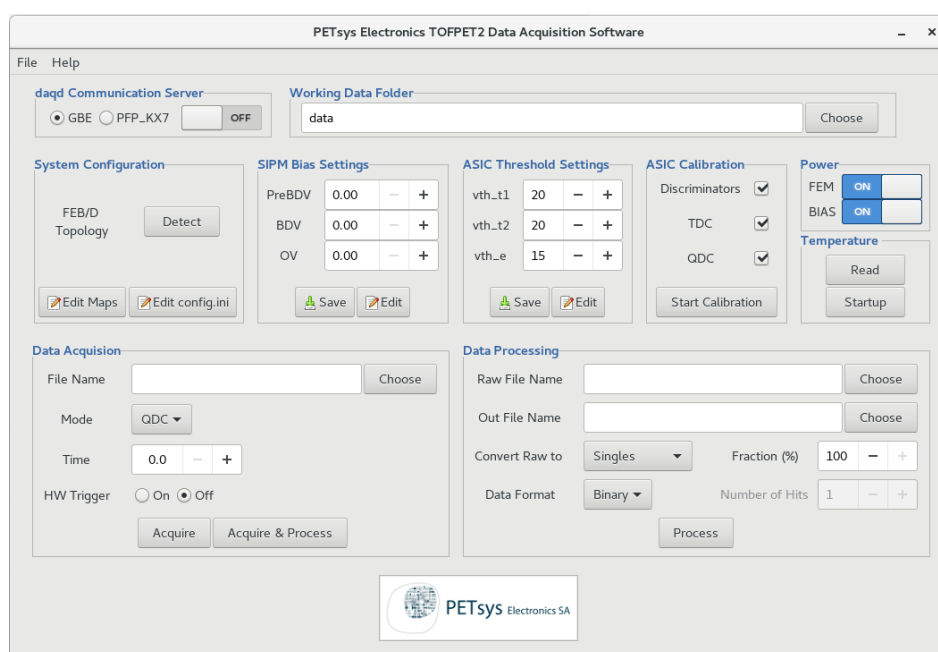


Figure 2.1: PETsys Electronics Data Acquisition Software: Graphical User Interface (GUI)

¹IMPORTANT NOTE: In order to avoid any damage to the hardware and ASICs, always power off the FEM boards (using the GUI FEM power switch or turning off the FEB/D) when plugging or unplugging any hardware and never perform data acquisitions with SiPMs under ambient light.

2.1 Starting the GUI

Since software version v2019.01, PETsys Electronics provides a Graphical User Interface (GUI) to control nearly all functionalities of the Data Acquisition Software. In order to start the GUI, just open a system terminal, navigate to the software base folder and type:

```
./start_gui
```

This will open the GUI (remember that if the terminal from which it was launched is closed, the GUI will also close), which is shown in Fig. 2.1.

2.1.1 Launching “daqd”

Before any communication with the hardware is attempted, “daqd” must be running. Information about the type of connection used for communication is required. In the section “daqd communication server”, choose the option “GBE” and turn the switch on:



The system will open a new terminal automatically and, if a successful communication is established with the FPGA, the initial output on “daqd” will read (the size of frame may be different than this example):

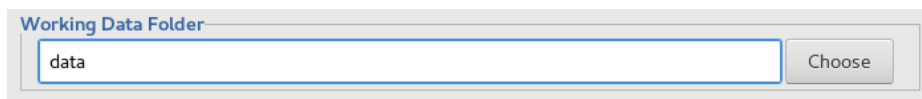
```
Size of frame is 16400
Got FPGA reply
FrameServer::startWorker called...
FrameServer::startWorker exiting...
UDPFrameServer::runWorker starting...
```

Otherwise, the GUI will show an error message. If this happens, the user should check that all cables and hardware are properly connected and powered up and retry. Since “daqd” is the communication server, the terminal in which it is running should not be closed at any time. Any other output in the “daqd” terminal/window after this initial message should NOT be taken into account, as this is for debug only. If an error occurs, a message will be shown either in the GUI itself or in the system terminal that was used to launch the GUI.

2.2 Configuration and calibration

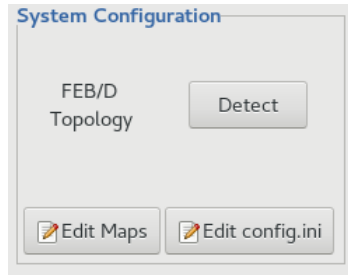
The concept behind the Data Acquisition Software data storage is that both calibration, configuration and acquisition data files are stored in the same folder, hereafter named Working Data Folder (see Sec. 3.1 for more details).

First step should be to create this folder, which can be done via the GUI. In the section “Working data folder”, just type the folder name to be created (“data” in this example) :



and press Enter (alternatively, the folder can be created using the “Choose” button, followed by “create folder” icon in the opened dialog). After confirmation, the folder is created and set as the current Working Data Folder.

Next step will be to detect the hardware (just the FEB/D) and save all required files into the Working Data Folder. This is done automatically by the GUI using the “Detect” button in the section “System Configuration”.



After pressing the “Detect” button, a window with the description of the detected FEB/D and HV DAC mezzanine type will show, asking if the user wants to save this configuration. The user should answer “yes”, which will copy and create some necessary files in the Working Folder Directory defining the system configuration. For Evaluation Kits shipped before June 2018, the corresponding HV DAC mezzanine is 64P and the user will be asked to enter the calibration file which was provided by PETsys Electronics.

Before acquiring any data, the user should check what is the recommended Breakdown Voltage and Overvoltage to bias the SIPMs that are connected. This information needs to be saved into a table to serve as reference for the software. PETSYS Electronics has provided its costumers with different types of individual 3x3mm SIPMs (coupled to 3x3x5mm LYSO crystals). The recommended settings for each of them are shown in Figs 2.2 and 2.3.

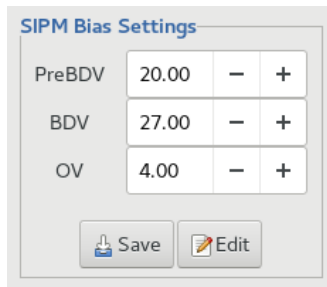


Figure 2.2: Bias Settings for Models KETEK PM3350T-S02, PM3325-WB-A0, PM3325-WB-B0

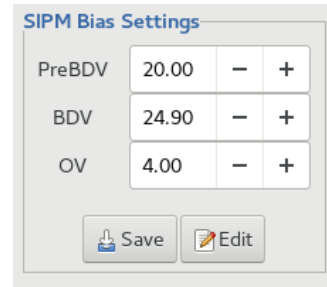
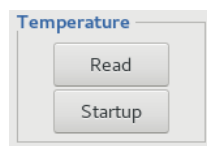


Figure 2.3: Bias Settings for Model KETEK PM3325-WB-D0 (Shipped after December 2018)

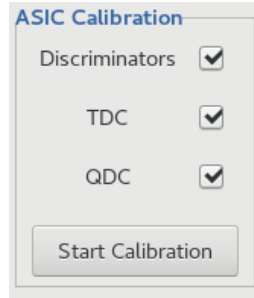
If using other SIPMs, set the “BD” to the Breakdown voltage of your SIPM and “OV” to the desired working overvoltage. “PREBD” should be set to a value about 20 % lower than the Breakdown Voltage. After setting the SIPM bias settings, these should be saved using the corresponding “Save” button on the bottom-left.

Some of the internal TOFPET2 ASIC circuitry, like the TDCs and QDCs, require calibration. However, before starting any calibration procedure, it is important to check for temperature stability at the ASIC level (since large temperature shifts can have an effect on TDC and QDC measured codes). In order to do so, just press the button “Startup” on the Section “Temperature”.



This will open a new window and the measured temperatures from all connected ASICs will be displayed every minute. When all temperatures are stable over one minute, the program will finish, indicating that the system is ready to acquire data. If no temperature stabilization is achieved in 10 minutes or more, the user must ensure that the external cooling system is properly installed and retry.

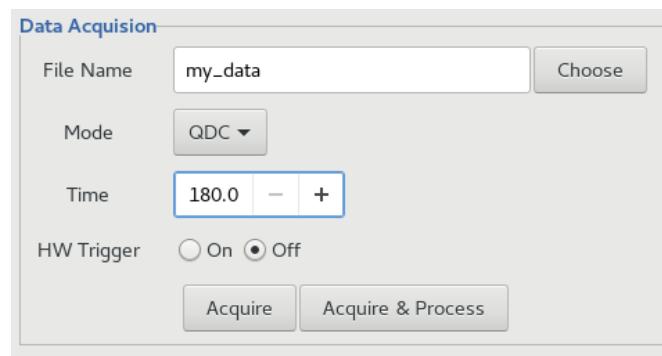
The user may then proceed to perform the calibration. After ensuring that all three options in the section “Calibration” are toggled, just press the button “Start Calibration”.



A prompt window will open requiring confirmation, and informing that this procedure can take some time to run. After confirming, a process window will open and run a suite of calibration scripts, taking more than 40 minutes to finish, depending on the computer and the number of FEMs connected in the system. Configuration tables and plots (1 plot per ASIC) will be saved in the previously created folder “data”. During the calibration procedure the discriminator thresholds will be saved to the default values: $vth_{t1} = 20$; $vth_{t2} = 20$; $vth_e = 15$. This means that, for the purpose of this chapter, the user doesn’t need to perform any action on the section “ASIC Threshold Setting”. Nevertheless, the possibility of changing the ASIC discriminator thresholds is documented in Sec. 4.1 and can only be performed after finishing the calibration.

2.3 Data Acquisition with a radioactive ^{22}Na point source

The user is now ready to acquire data with the evaluation kit. To do so, type the name of the desired output file in the “Data Acquisition” section:



“Mode” should be set to “QDC”. The acquisition time depends on the activity of the ^{22}Na source, but 180 seconds should be enough for a ^{22}Na source of about $20\ \mu\text{Ci}$. If using lower activity source, then the acquisition time should be increased accordingly. “HW Trigger” can be set either “on” or “off”. After pressing “Acquire”, a process window will open printing the amount of data frames and time acquired. At the end of the acquisition, a small report will be printed containing the number of acquired data frames and the number of registered events. If the later number is 0, the user must make sure that the correct bias voltage is being set and that all boards, including SIPMs are tightly connected.

2.3.1 Coincidence events and Coincidence Time Resolution

If the ^{22}Na source was placed in between two SIPM detectors, one can process the data and filter it for coincidence events (events that correspond to the detection of two Gamma photons within a short time window and in different geometrical regions). To do so, set the section “Processing data” as follows (the name of the input and output files are by default automatically suggested based on the raw data acquisition file name and final output event type, chosen from the drop down menu of “Convert Raw to”, and the final data format)

The screenshot shows a 'Data Processing' window with the following controls:

- Raw File Name:** A text box containing 'my_data' and a 'Choose' button.
- Out File Name:** A text box containing 'my_data_coinc.ldat' and a 'Choose' button.
- Convert Raw to:** A dropdown menu set to 'Coincidences'.
- Fraction (%):** A numeric input field set to '100' with minus and plus buttons.
- Data Format:** A dropdown menu set to 'Binary'.
- Number of Hits:** A numeric input field set to '1' with minus and plus buttons.
- Process:** A central button at the bottom.

and press the button “Process”. A new window will open and after processing the raw data (which can take a few seconds), it will print a report where the final number will correspond to the number of coincidence events found. The output file “data/my_data_coinc.ldat” will contain a list (in this case, in binary format) containing the channel ID, time of arrival and energy (QDC measurement) for each of the two events of all coincidences. The user can then run the script “psDrawCTR.C” to plot the energy spectra on both channels as well as the coincidence time difference distributions. This is done outside the GUI, by opening a system terminal, navigating to the software folder and typing:

```
root -l psDrawCTR.C+' ("data/my_data_coinc")'
```

An example of the output is presented in fig. 2.4. The script plots three histograms: two coincidence energy distributions, one for each channel with the higher number of coincidence events. For each energy distribution, a gaussian fit is performed in order to determine the mean value and dispersion of the photopeak corresponding to the 511 keV gamma photons. In the middle panel, it is plotted the histogram for the time of arrival difference between the two photons for every coincidence event. To obtain this distribution, only events with energies compatible with a 1-sigma energy window around the photopeak centers are selected. The fit of a gaussian model to this distribution determines the time of flight variable and also the Coincidence Time Resolution (CTR) for that channel pair. In this example, the CTR is 85.7 ps sigma, which corresponds to 202 ps FWHM.

Hopefully, the present chapter allowed the user to quickly obtain data and CTR results from your new Evaluation Kit. The next chapters will describe in more detail the routines and procedures of calibration, data acquisition and data processing.

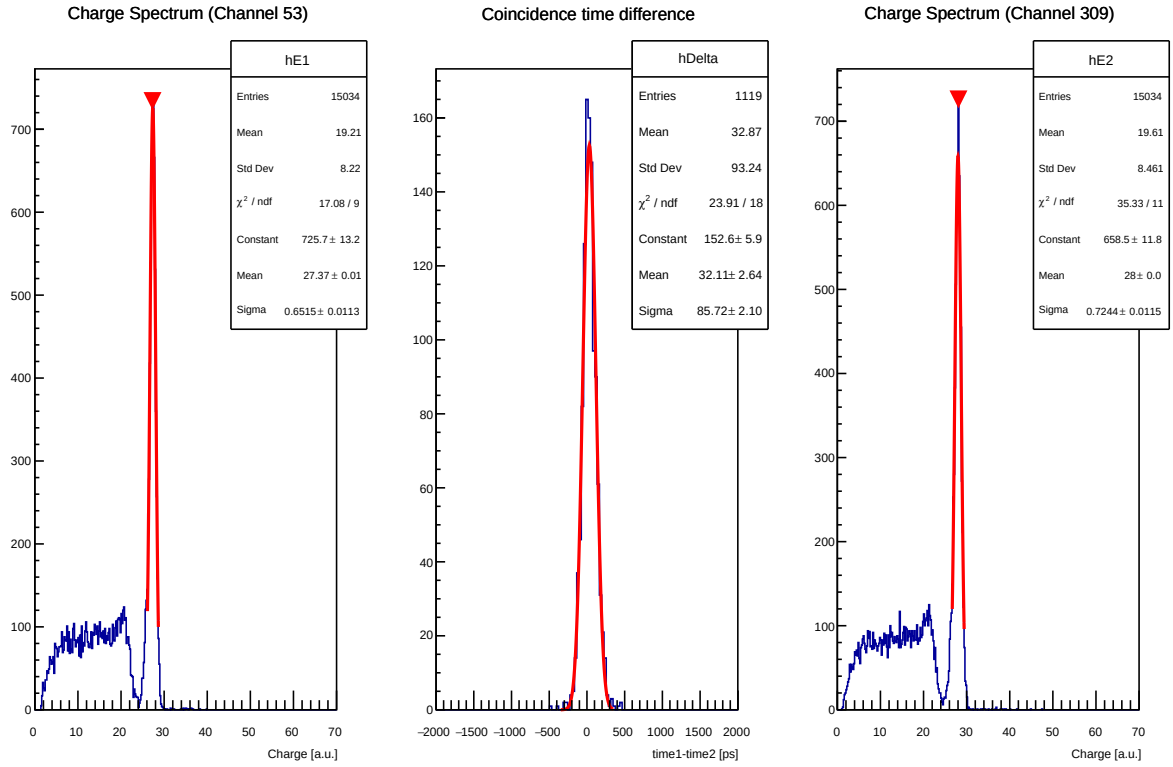


Figure 2.4: Example for the output of “psDrawCTR.C”, showing spectra (from QDC) and coincidence time resolution (CTR) for a channel pair reading two KETEK PM3325-WB SIPMs coupled to 3x3x5mm LYSO crystals.

Chapter 3

Reference Configuration File and Calibration procedure

NOTE: PETsys Electronics TOFPET2 data acquisition software is essentially based in Python scripts or C++ compiled executable functions, which can be called from any system terminal (command line approach). The available GUI manages to wrap all, or almost all possible commands in a self-contained interface, which can be used standalone. Nevertheless, the user may still want to use the command line approach, which is documented in the next three chapters, together with the corresponding GUI usage (when applicable).

In order to operate properly, TOFPET2 ASICs and external HV DACs require some level of configuration and calibration. Configuration files are required to acquire data and calibration files are used to obtain correct time and charge stamps for each event, from the raw data provided by the ASICs. In the present software framework, all the files and parameters required for configuration and calibration are referenced in a single file: the Reference Configuration File - “config.ini”. It is a required argument for almost all command line software applications, either for data acquisition or data processing.

An example of such file is provided in the software package and is show in listing 3.1.

Listing 3.1: config.ini

```
1 [main]
2 bias_calibration_table = %CDIR%/bias_calibration.tsv
3 bias_settings_table = %CDIR%/bias_settings.tsv
4
5 tdc_calibration_table = %CDIR%/tdc_calibration.tsv
6 qdc_calibration_table = %CDIR%/qdc_calibration.tsv
7 disc_calibration_table = %CDIR%/disc_calibration.tsv
8
9 acquisition_mode_table = %CDIR%/acquisition_mode_settings.tsv
10 #energy_calibration_table = %CDIR%/energy_calibration.tsv
11 disc_settings_table = %CDIR%/disc_settings.tsv
12
13 channel_map = %CDIR%/map_channel.tsv
14 trigger_map = %CDIR%/map_trigger.tsv
15
16 [hw_trigger]
17 type = builtin
18 threshold = 1
19 pre_window = 3
20 post_window = 15
21 coincidence_window = 3
22 single_fraction = 0
23
24 [asic_parameters]
25 global.disc_lsb_T1 = 60
```

The file is divided in two sections. The “[main]” section (lines 2-12) defines the required configuration and calibration files, as well as map tables. These are:

- “bias_calibration_table” - Calibration table for the bias voltage DAC present on the FEB/D. Allows to provide correct bias voltage to SIPMs (up to 10 mV). The file is created with a software script from a previous calibration file provided by PETSys Electronics or from information saved in the EEPROM of the Bias Mezzanine board (systems shipped after June 2018).
- “bias_settings_table” - Configuration table containing 4 voltages (in Volts) to be defined for each Bias DAC channel of the Bias Mezzanine and applied to the SIPMs that are connected to the front-end modules when acquiring data. The 4 voltages are: “Offset” is an offset voltage to be added in order to compensate the voltage drop at the input of each ASIC channel. For TOFPET2 ASIC this value should be set to 0.75V; “Pre-breakdown”, a value below the breakdown voltage of the SIPM to be used by some calibration applications to acquire data with test pulses while having the SIPM capacitance as close to normal operation as possible; “Breakdown” is the breakdown voltage of the SIPM which serves as reference to set the “Overvoltage” to be applied in order to bias the SIPMs when acquiring data. It is defined by the user, and a script is provided to create this table, for the case where all channels provide the same value (see below).
- “tdc_calibration_table” - Calibration table for the time-to-digital converters (TDC) present in each channel. Allows to correct the very low level of non-linearity in the correspondence between TDC codes and reference clock phase. Created by a calibration application provided by PETSYS Electronics.
- “qdc_calibration_table” - Calibration table for the charge integrator (QDC) present in each channel. Created by a calibration application provided by PETSYS Electronics.
- “disc_calibration_table” - Calibration table for the 3 discriminators present in each TOFPET2 channel: t1, t2 and E. It contains the information for the baseline and noise level for each of the 3 discriminators, as well as dark count rate at different thresholds. Table obtained by running a calibration script provided with the software package.
- “disc_settings_table” - A configuration table that defines the threshold levels for each of the three discriminators. It is defined by the user, but a script is also provided to create this table in the case where all channels have the same value on a given discriminator.
- “channel_map” - A map table used by the post-processing applications to obtain information relative to a given system Channel ID. It can be defined by the user, since some of its elements depend on the system geometry, but a template with “dummy” values regarding SIPM physical geometry and pixel position can be created with a provided software function.
- “trigger_map” - A map table used by the the hardware trigger filter and the software post-processing applications to identify the possible channel ID regions where to search for grouped events or coincidence events. A simple version is also created with the provided software function.

All the files described above consist of TAB separated value columns allowing the user to easily read or modify them. All the files described above should be located in the the same folder as the reference configuration file, usually the Working data folder where the acquired data will be stored. If using the GUI, all these files are created when saving the configuration detected with the “Detect” button or during the calibration performed with the “Start Calibration” button.

In the section “[hw_trigger]”, the user can change the in-built hardware trigger filter of events, as described in Sec. 4.2.2.

The third section of the reference configuration file (“[asic_parameters]”) is reserved to change the default values of the TOFPET2 ASIC configuration parameters. For typical PET applications with LYSO crystals, PETSYS Electronics only recommends to change t1 LSB parameter (“global.disc_lsb.T1”). It should be set accordingly to the gain of the SIPM and the pulse shape. According to the tests done by PETSYS Electronics, a proper working value is 57 (which corresponds to 5mV per DAC unit) for most SIPMs except for KETEK PM3325-WB, for which it can be set to 60 (corresponding to about 2.5mV per DAC unit). This choice implies that the amplitude for one photoelectron corresponds to about 5 DAC units of the t1 threshold.

If the user wants to change other ASIC configuration parameters, please refer to the document “PETSys TOFPET 2 ASIC - Datasheet.pdf” where you can find information on parameter names and functionalities, as

well as acceptable value ranges. The parameter values can be modified in this section, by defining the prefix “global.parameter_name” or “channel.parameter_name” for global ASIC parameters or channel specific parameters (in the later case all channels will be set to the same value).

The Reference Configuration File can be opened and edited directly from the GUI, with the button “Edit config.ini”. After performing any change, it is important to save the file using the same file name: “config.ini”.

3.1 Configuration and calibration

Each row on the above mentioned files usually corresponds to a different channel or TAC (either ASIC channel) or Bias DAC channel, depending on the file. The ID of such channels depends on which port of the FEB/D the TOFPET2 ASICs are connected. This means that the configuration tables are valid if and only if they were obtained for a given configuration (connection of Front End Modules to FEB/D ports) and this configuration is not changed. It is IMPORTANT to create new configuration files whenever this is not verified, or when using the hardware for the first time (as in the previous chapter of this guide) or when the operating temperature of the ASIC has significantly changed (operating temperature variations of more than 5°C, measured by the temperature sensors, require new calibration data). To do so, in command line approach, use the template script provided with the software and shown in listing 3.2. As described in Chapter 2, it is recommended to create a new Working Data Folder whenever performing a new calibration. Also, before running any script, please ensure that “daqd” server is running, either by using the GUI switch as described in Sec. 2.1.1 or by typing in a separate system terminal:

```
./daqd --socket-name=/tmp/d.sock --daq-type=GBE
```

Listing 3.2: configuration.template.sh

```
1  #!/usr/bin/sh
2  DATA_DIR=$1
3  CONFIG_FILE=${DATA_DIR}/config.ini
4
5  ./acquire_threshold_calibration --config ${CONFIG_FILE} -o ${DATA_DIR}/
   disc_calibration
6  ./process_threshold_calibration --config ${CONFIG_FILE} -i ${DATA_DIR}/
   disc_calibration -o ${DATA_DIR}/disc_calibration.tsv --root-file ${
   DATA_DIR}/disc_calibration.root
7
8  ./make_simple_disc_settings_table --config ${CONFIG_FILE} --vth_t1 20 --
   vth_t2 20 --vth_e 15 -o ${DATA_DIR}/disc_settings.tsv
9
10 ./acquire_tdc_calibration --config ${CONFIG_FILE} -o ${DATA_DIR}/
   tdc_calibration
11 ./acquire_qdc_calibration --config ${CONFIG_FILE} -o ${DATA_DIR}/
   qdc_calibration
12
13 ./process_tdc_calibration --config ${CONFIG_FILE} -i ${DATA_DIR}/
   tdc_calibration -o ${DATA_DIR}/tdc_calibration
14 ./process_qdc_calibration --config ${CONFIG_FILE} -i ${DATA_DIR}/
   qdc_calibration -o ${DATA_DIR}/qdc_calibration
```

This script “configuration.template.sh” runs a series of applications that create the three calibration tables described above (“disc_calibration.table”, “tdc_calibration.table” and “qdc_calibration.table”).

Before running the calibration script, if not using the GUI (command line approach), the user should create a SIPM bias voltage table (“sipm_bias_table”) for the SIPMs that are plugged in the test boards. For example, if using KETEK PM3325-WB-D0 SIPM (delivered with the most recent versions of the PETSYS Electronics Evaluation Kit), the user can create a new table by typing:

```
./make_simple_bias_settings_table --config data_folder/config.ini
--offset 0.75 --prebd 20 --bd 24.9 --over 5 -o data_folder/bias_settings.tsv
```

In this example, the pre-breakdown voltage is being set to 20V, the SIPM breakdown voltage to 24.9V and the overvoltage to 5V. Due to a 750mV DC voltage at the input of each TOFPET2 ASIC channel, the offset voltage should always be set to 750mV. Operational SIPM bias voltage in this example will be $24.9\text{V} + 5\text{V} = 29.9\text{V}$. This value will be set by the HV DAC in all its channels, whenever acquiring SIPM data. Pre-breakdown value is used when calibrating the ASIC. For different SIPMs (or operating the SIPM at a different overvoltage), the user should create a different file accordingly.

It is also important to create a bias calibration file, which will be used to insure that correct voltages are provided by the Bias DAC channels. In order to do so (for hardware shipped after June 2018), type:

```
./make_bias_calibration_table -o data_folder/bias_calibration.tsv
```

This command will read the calibration voltages measured by PETsys Electronics and stored on each BIAS-16P mezzanine EEPROM and create a calibration file for the entire readout system, that can be used by the software programs when requested.

If the user has an older Bias Mezzanine version (shipped before June 2018), then the calibration file must be created from the files provided by PETsys Electronics, for example:

```
./make_bias_calibration_table -o data_folder/bias_calibration.tsv --port 0  
--slave 0 --filename hvdacMezzXX_ad5535_cal.tsv
```

With both “bias_calibration.table” and “bias_settings.table” created and referenced in the Reference Configuration File, the ASIC(s) configuration script can then be run with the following command (assuming the user has renamed the script name from “configuration.template.sh” to “configuration.sh”):

```
sh configuration.sh data_folder
```

If using the GUI, all commands documented above are applied automatically when using the “Save” button in the “Bias Settings” section and then pressing the “Start Calibration” button in the “Calibration” section.

3.1.1 Discriminator calibration

Lines 5 and 6 in listing 3.2 correspond to the data acquisition and processing for discriminator calibration. The data acquisition consists in a scan of the discriminator threshold in order to find the baseline and noise estimation (with the SIPM bias voltage set to a value below breakdown voltage). The script also performs a dark count scan, by performing the same threshold scan but this time with the SIPM biased at operational voltage. The data is processed in the last line which also allows to obtain the calibration table “data/disc_calibration.tsv”. Summary PDF files, will be created for each ASIC, containing the values for the amplifier’s adjustment DAC, baseline and noise per channel. These should have data on all channels without a significant dispersion, as exemplified in Fig. 3.1.

3.1.2 TDC calibration

The 10th line in listing 3.2 launches a data acquisition where test pulses trigger the TDCs of all connected ASICs at different known phases relative to the reference clock. Line number 13 processes and fits the obtained data in order to determine the calibration function mapping measured TDC codes and time. It writes the parameters for those functions in the TDC calibration table, in this example the resulting file is “data/tdc_calibration.tsv”. The application will also create a series of ROOT files (containing all the histograms and data fits) and PDF files (summarizing the time resolution obtained after calibration with the test pulse triggered data), one for each ASIC connected. The summary plot for each ASIC should resemble the one on Fig. 3.2, with data on all 256 TACs (4 TAC per channel) for each discriminator and the time resolution after calibration on all of them below 0.01 clock period (RMS). If a significantly different plot is obtained for any ASIC, the user should contact the PETSYS Electronics support team.

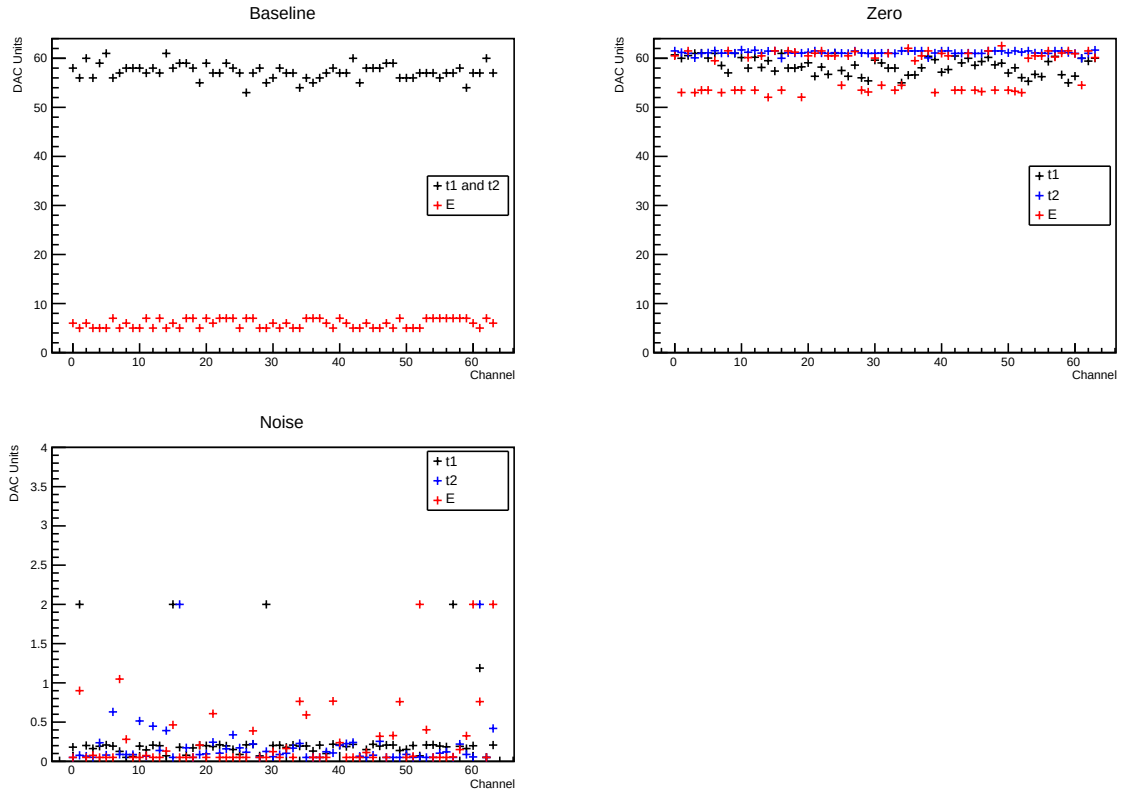


Figure 3.1: Discriminator calibration summary plots for one ASIC. On the top left plot, the value of the amplifier's adjustment DAC after convergence, per channel. On the top right plot, the value of the threshold level which corresponds to the amplifier's baseline, in DAC units, per channel. The bottom left shows the noise level measured per channel, again in DAC units. Different colours represent data from the 3 discriminators: t1 (black), t2 (blue) and E (red).

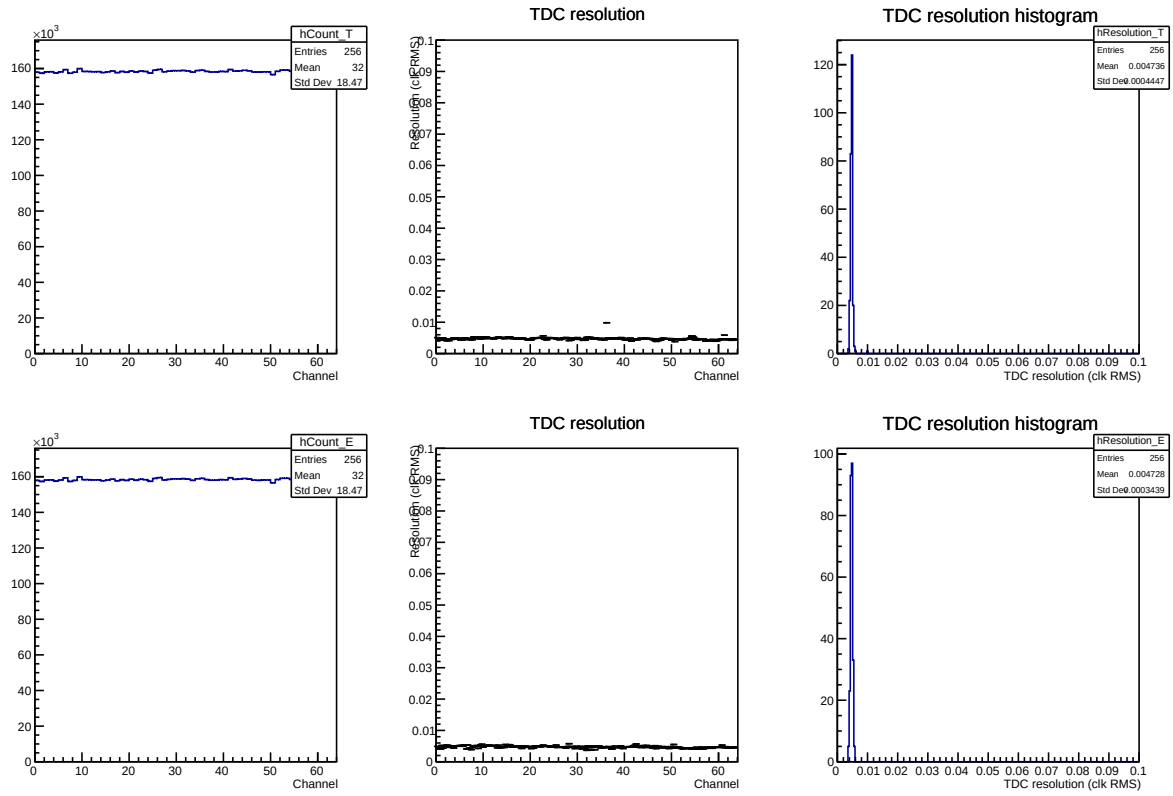


Figure 3.2: TDC calibration summary plots for one ASIC. The upper plots show the number of test pulse events used for calibration (left), the distribution per channel of the obtained TAC resolution after calibration (middle) and the overall distribution of resolutions obtained after calibration (right), for the first discriminator (t1 in default trigger mode). The plots on the bottom are the same but for the second discriminator (E in default trigger mode).

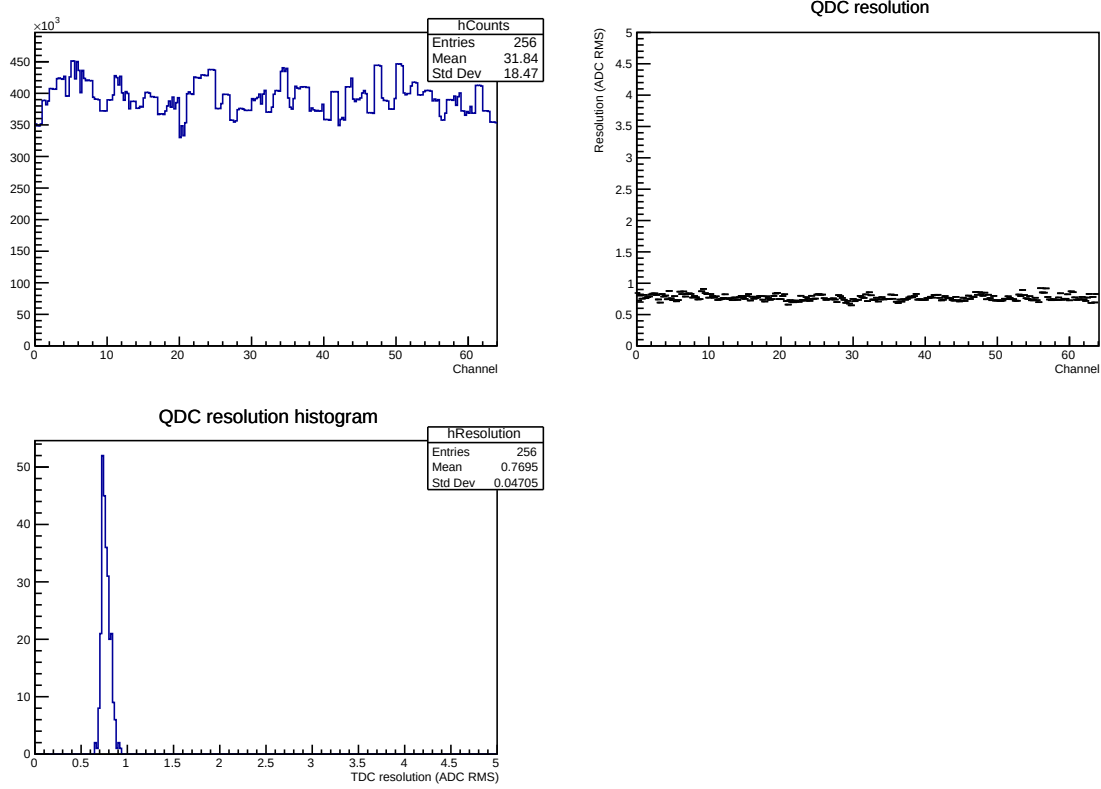


Figure 3.3: QDC calibration summary plots for one ASIC. On top left, the number of events used to calibrate each channel. On top right, the distribution for all channel and TACs of the average resolution after calibration. On top left, the histogram of all average resolutions for the ASIC.

3.1.3 QDC calibration

Line number 11 of listing 3.2 acquires data for calibration of the charge integrator (QDC), by varying the integration window of test pulse driven events. The data is then processed in line number 14, in order to determine the offset current to be removed when integrating the charge of an event. A summary plot (Fig. 3.3) is created for each ASIC, presenting with the number of test pulse events used for the calibration and the measured resolution after calibration (difference between measured values by the QDC and the best-fit model), to ensure that all channels had sufficient data to calibrate correctly. The measured resolutions should be under 3 ADC RMS, otherwise the user should contact the PETSYS Electronics support team.

3.1.4 QDC charge measurement linearisation / Energy Calibration (Optional)

The ASIC QDC pre-amplifier, designed and optimized for measuring the 511 keV gamma rays with LYSO scintillating crystal and SiPM photosensors, has a slightly non-linear response over its nominal dynamical range. When measuring the energy of gamma rays with SiPM photodetectors and scintillating crystals there are thus two effects contributing to the non-linearity of the energy measurement: SiPM saturation and the non-linearity of the TOFPET2 ASIC charge measurement. One can calibrate the overall non-linearity by using several gamma-ray emitting isotopes with photopeaks of different known energies. For example, using ^{22}Na (511 keV), ^{137}Cs (32 and 662 keV) and ^{133}Ba (31 keV, 81 keV and 356 keV), one can effectively linearise the QDC/SIPM by fitting the Energy (E) vs measured charge (Q) data in a one-to-one SIPM to crystal coupling with an appropriate parametric function, such as:

$$E = P_0 \cdot p_1^{Q^{P_2}} + P_3 \cdot Q - P_0 \quad (3.1)$$

In many situations, the user may not have such sources available in order to calibrate every ASIC channel independently. However, PETsys Electronics has certified that using the above function with $(P_0, P_1, P_2, P_3) =$

(8.00000, 1.04676, 1.02734, 0.31909) allows to approximate the non-linear response of the ASIC's QDC for all channels, up to a level of 1-2% on the energy resolution of the 511 keV photopeak. In order to use this universal linearisation, the user must first create a simple table using the command (assuming the working data folder is "data"):

```
/make_simple_energy_calibration_table --config data/config.ini  
-o data/energy_calibration.tsv
```

The created table can be used to linearise/calibrate the charge measurement when processing data (as in Chapter 5). To do so, just open the reference configuration file "config.ini", uncomment the line referring to "energy_calibration_table" and save.

3.2 Map tables

Two tables are required for the data processing applications, described in Chapter 4.

3.2.1 Channel map table contents

The channel map table is a tab separated table text file, where each row has the information related to a given ASIC channel. The 10 columns are:

- portID: The DAQ port ID where the FEB/D is connected (see Annex A).
- slaveID: The slave ID of the FEB/D when connecting multiple FEB/Ds in daisy chain (see Annex A).
- chipID: The ASIC ID, related to the position where it is connected in the FEB/D (see Annex A).
- channelID: The channel ID inside the ASIC (see Annex A).
- regionID: Trigger region ID number (e.g., 0, 1, 2...).
- xi: Pixel index number in the X direction (0, 1, 2, ...).
- yi: Pixel index number in the Y direction (0, 1, 2, ...).
- x: Pixel spatial position in the X direction (in mm).
- y: Pixel spatial position in the Y direction (in mm).
- z: Pixel spatial position in the Z direction (in mm).

The trigger region ID allows ASIC channels to be grouped in geometrical regions (by attributing them the same trigger region ID) for selection of multi-hit and coincidence events when processing data the software will only consider as coincidence events those that occur in trigger regions with different IDs as defined in the trigger map (as described below).

The pixel index numbers in the X and Y direction are simply tagged to the event data when writing to the output ROOT data format and can be useful for data analysis. For example, these can be useful if the user wants to construct a flood map. The pixel spatial position values are used by the data processing software ("convert_raw_to_group" and "convert_raw_to_coincidence") to group events into multi-hits (groups of events produced by the same Gamma photon) and can also written to the output ROOT data format for analysis (see Chapter 4).

3.2.2 Trigger map table contents

The trigger map table is a table with three columns, where the first 2 columns define a trigger region pair and purpose of such trigger pair in the software post processing: coincidence (C) or multi-hit group (M). Typically, a multi-event (or multi-hit) group is composed of events occurring in the same trigger region, while a coincidence event requires two multi-hit groups to occur in different trigger regions.

3.2.3 Creating a simple map

PETSYS Electronics provides a program to create a simple version of such maps, under the principle that all channels connected to ports F1 and F2 (or F3 and F4, etc...) of the FEB/D belong to the same trigger region. In order to use it just type:

```
./make_simple_channel_map -o data_folder/map --port 0 --slave 0 --trigger-port  
0
```

This application will create both the channel and trigger maps, filling the first 5 columns. For the other 5 columns that correspond to the geometrical position of the detectors connected to each channel, since those are system dependent, they should be filled by the user reflecting its system configuration. Nevertheless, even if these values are left as 0's, all data processing applications can still run correctly.

If using the GUI, the simple maps are created when using the “Detect” button in the “System configuration” section and after saving the configuration. They can be edited for customisation with the “Edit Maps” button on the same section. Again, after editing, they should be saved using the same file names.

Chapter 4

Data acquisition

The acquisition application software consist, mostly, of a series of Python scripts and libraries which implement 3 main functions.

- Configure the hardware.
- Acquire raw data.
- Store raw data into files.

4.1 Data acquisition template

A basic data acquisition Python script is provided for reference and shown in listing 4.1. This Python program performs the following basic tasks (after parsing the arguments, which ends at line 14):

- Read and store configuration and calibration data from the files defined in the reference configuration file
- Connect to “daqd” and initialize the hardware.
- Load the hardware configurations and define operational bias voltage (from config file)
- Modify the ASIC configuration as needed (in this case setting the two configuration options related to TOT or QDC).
- Upload the ASIC configuration into the hardware.
- Open the output raw data file for writing.
- Acquire data during a period of time.
- Set the HV bias voltage to 0 on all channels and exit.

Listing 4.1: acquire_sipm_data_basic

```
1  #!/usr/bin/env python
2  # -*- coding: utf-8 -*-
3
4  from petsys import daqd, config
5  from copy import deepcopy
6  import argparse
7
8  parser = argparse.ArgumentParser(description='Acquire_data_for_TDC_calibration')
9  parser.add_argument("--config", type=str, required=True, help="Configuration_file")
10 parser.add_argument("-o", type=str, dest="fileNamePrefix", required=True, help="Data_filename_
    (prefix)")
11 parser.add_argument("--time", type=float, required=True, help="Acquisition_time_(in_seconds)")
12 parser.add_argument("--mode", type=str, required=True, choices=["tot", "qdc"], help="
    Acquisition_mode_(ToT_or_QDC)")
13 parser.add_argument("--enable-hw-trigger", dest="hwTrigger", action="store_true", help="Enable
    the_hardware_coincidence_filter")
```

```

14 args = parser.parse_args()
15
16 systemConfig = config.ConfigFromFile(args.config)
17
18 daqd = daqd.Connection()
19 daqd.initializeSystem()
20 systemConfig.loadToHardware(daqd, bias_enable=config.APPLY_BIAS_ON, hw_trigger_enable=args.
    hwTrigger)
21
22 qdcMode = args.mode == "qdc"
23
24 asicsConfig = daqd.getAsicsConfig()
25 for ac in asicsConfig.values():
26     gc = ac.globalConfig
27     for cc in ac.channelConfig:
28         cc.setValue("trigger_mode_1", 0b00)
29
30         if not qdcMode:
31             cc.setValue("qdc_mode", 0)
32             cc.setValue("intg_en", 0)
33             cc.setValue("intg_signal_en", 0)
34         else:
35             cc.setValue("qdc_mode", 1)
36             cc.setValue("intg_en", 1)
37             cc.setValue("intg_signal_en", 1)
38
39
40 daqd.setAsicsConfig(asicsConfig)
41
42 daqd.openRawAcquisition(args.fileNamePrefix, qdcMode=qdcMode)
43 daqd.acquire(args.time, 0, 0);
44
45 systemConfig.loadToHardware(daqd, bias_enable=config.APPLY_BIAS_OFF)

```

4.2 Data acquisition procedure

Before acquiring any data, the user needs to define the threshold values for each of the ASIC's discriminators. These values are defined in a table which is referenced in the "disc_settings_table" field of the reference configuration file. For most SIPMs tested by PETSYS Electronics in the default triggering mode, we recommend a threshold of 20 DAC units above the baseline value for t1 discriminator, the same for t2, and a value of 15 DAC units above the baseline for the E discriminator. The user can create a table that sets the same threshold value for all the connected channels by running the "make_simple_disc_table" application. For the example of the recommended settings, this can be done by typing:

```

./make_simple_disc_settings_table --config data_folder/config.ini --vth_t1 20
--vth_t2 20 --vth_e 15 -o data_folder/disc_settings.tsv

```

The above command will write the output into the table file named "disc_settings.tsv", which is referenced in the configuration file. The user is free to choose other values for the thresholds, taking into account that the values in the table correspond to ADCs above the baseline and the DACs are 6-bit, ranging from 0 to 63.

If using the GUI, the above command is executed when using the "Save" button in the "ASIC Threshold Setting" section. The created file can be edited with the "Edit" button in the same section, for example, to set different threshold values to different channels. Here again, after editing, the file should be saved using the same file name.

With the threshold, SIPM bias and calibration tables defined and referenced in the configuration file, the user can acquire data from the SIPMs and ASICs. It is recommended to use the script "acquire_sipm_data". The script can be ran as follows:

```

./acquire_sipm_data --config data_folder/config.ini --mode qdc --time 60
-o data_folder/data_prefix

```

Each required (or optional) argument is preceded by a corresponding flag. The user may check all flags and argument descriptions by typing “-help” anywhere after “./acquire_sipm_data”.

In the above data acquisition example, the first flag is “-config” which is used to specify the reference configuration file described in chapter 3. Other required flag is “-mode”, which can only take three options: “qdc”, “tot” or “mixed”. In “qdc” mode, the output energy measurement for each pulse is obtained by linear charge integration. PETSYS Electronics recommends to use this mode (for better resolution and linearity), except for very large signals that could saturate the integrator, or small amplitude signals (less than 10 photo-electrons). If using “tot” mode, the energy measurement is obtained from Time-over-threshold, using the E and t1 discriminators time stamps (non-linear measurement). The “mixed” mode option allows to acquire data with a part of the system’s channels acquiring in “qdc” mod and the other part in “tot” mode. In order to use this option, the user needs to manually create and save a table with the name “acquisition_mode.table.tsv” in the working data folder, defining a mode for each of the system’s channels. An example of the format of such file is presented below:

```
#portID slaveID chipID channelID mode
0        0        0        0        qdc
0        0        0        1        qdc
0        0        0        2        qdc
0        0        0        3        qdc
...
0        0        1        0        qdc
0        0        1        1        qdc
0        0        1        2        qdc
0        0        1        3        qdc
...
```

All acquisition modes are selectable in the GUI within the “Mode” drop down list.

The “-time” flag sets the amount of time of the data acquisition, 60 seconds in the above example. If using the GUI, it can be set in the “Time” entry box.

The “-o” flag sets the output data file prefix. In this example, the script will produce 2 files named “data_prefix.rawf” and “data_prefix.idx” and placed inside the folder “data_folder”. The file “data_prefix.rawf” contains the raw data in compact binary format. Its exact structure is not documented in this guide since the time information obtained directly from the ASICs, as well as the energy measurement from the integrator require correction using calibration on the post processing level, as previously described. The “data_prefix.idx” file is an auxiliary text file containing information on the range of events for each of the variables “step1” and “step2”, which is only relevant if the user is performing parameter scans as described below.

If using the GUI, the acquisition is performed when pressing the “Acquire” button, and it will use the corresponding options as they are selected. If pressing the “Acquire & Process” button, the same applies, but immediately after finishing the raw data acquisition it processes it according to the options in section “Data Processing” (see chapter 5 for details on data processing).

4.2.1 Parameter scan

The script “./acquire_sipm_data” allows to acquire data sequentially while varying up to 2 key parameters (grid scan). This can be useful in order to help the user to determine an optimal performance configuration. The operational parameters available for scanning are:

- “OV” - the SIPM Bias Overvoltage.
- “vth_t1” - t1 discriminator threshold, in DAC units relative to the channel baseline.
- “vth_t2” - t2 discriminator threshold, in DAC units relative to the channel baseline.
- “vth_e” - e discriminator threshold, in DAC units relative to the channel baseline.

- “disc_lsb_t1” - t1 discriminator DAC LSB. Its value allows to change the dynamic range for the t1 discriminator DAC.

In order to perform a parameter scan, the user needs to activate the optional flag “--param-table” and provide a table containing 1 or 2 TAB separated columns, where the header (first line) contains the identification of the parameter(s) to be varied and the next lines contain the values to be scanned. For example, if the user wanted to obtain results with SIPM overvoltages of 2V, 2.5V, 3V, 3.5 and 4V, while also varying the “vth_t1” discriminator value for a set of possible values, it could type:

```
./acquire_sipm_data --config data_folder/config.ini --mode qdc --time 60
-o data_folder/data_scan_prefix --param-table param_table.txt
```

Where the content of the file “param_table.txt” would be:

Listing 4.2: param_table.txt example

OV	vth_t1
2	10
2.5	20
3	30
3.5	40
4	50
	60

In this example, the script would then acquire for 60 seconds for each of the 30 possible parameter combinations, while writing the output raw data into the same file “data_scan_prefix.rawf”. The “data_scan_prefix.idx” would contain 30 lines with the information of the first and last event corresponding to a given parameter combination. Naturally, it is possible to define only one column for a single parameter scan. For the moment, the parameter scan feature is not implemented in the GUI, only in command line.

4.2.2 Hardware coincidence trigger

Although not used in the above example, the acquisition script has an additional optional flag “--enable-hw-trigger”. If this flag is used (no other argument required), the hardware (FPGA) will filter events based on their time of arrival and channel ID (using the same trigger regions defined in the template trigger maps described above). The parameters defining the trigger can be set in the “[hw_trigger]” section of the Reference Configuration File (“config.ini”) and are:

- “type” - The type of hardware filter to be implemented. At the present moment, only the option “builtin” is available.
- “threshold” - A raw energy threshold, which only works if the acquisition is in QDC mode (in DAC units, before calibration).
- “pre_window” - If a coincidence event is found, this variable sets the pre-event time window of accepted events, in clock period units (from 0 to 15).
- “post_window” - If a coincidence event is found, this variable sets the post-event time window of accepted events, in clock period units (from 0 to 127).
- “coincidence_window” - This variable sets the coincidence time window, in clock period units (from 0 to 3).
- “single_fraction” - Single event sampling switch, adding up to the coincidence filtered data. If set to 0, the single event sampling is disabled. If set to 1, data from 10 clocks period in each 1025 clock period is accepted. If set to 2, data from 20 clocks period in each 1025 clock period is accepted. If set to 3, data from 50 clocks period in each 1025 clock period is accepted. If set to 4, data from 100 clocks period in each 1025 clock period is accepted.

If using the GUI, this filter can be toggled “on” or “off” in the “HW Trigger” option. The hardware trigger parameters can be changed from the GUI by using the “Edit config.ini” button, changing the values and saving the resulting file without changing its name.

4.3 Temperature reading and stability

In order to monitor the temperature of your boards, the user can type (or use the “Read” Button in the GUI “Temperature” section):

```
./read_temperature_sensors
```

This will print a report with the values of all the measured temperatures, ordered with respect to the FEB/D port where the ASIC Boards are plugged. Please note that in order to obtain a correct reading, the switches sw1 and sw2 on the FEB/D motherboard should be properly configured, mapping the ports that are being used to connect your TOFPET2 ASIC Test Boards, as described in the “Evaluation Kit - Hardware User Guide”, page 12. However, if using an Evaluation kit with FEM64 instead of Test boards, there is no need to configure the switches.

IMPORTANT: After any power cycle of the FEB/D board, the first run of the acquisition script will configure the ASICs (for the first time), causing a rise in temperature that can affect the measured time and energy stamps. In order to avoid this effect, we recommend to run the following program (in the GUI, use the “Startup” Button in “Temperature” section) before the first data acquisition after a FEB/D power cycle:

```
./startup
```

The script will configure the ASICs with a default configuration, and measure the temperature from the sensors closest to the ASICs every minute. When all temperature are stable (up to 0.25°C) for 1 minute, the program will exit, indicating that the system is ready to acquire data. If no temperature stabilization is achieved in 10 minutes or more, the user must verify the air flow or cooling system and retry.

4.4 Other utility applications

As set of applications designed for operational utility is available in the software package. For instance, in order to check the system status at any time, the user may run:

```
./init_system
```

This program will print on screen the detected hardware and its IDs: (port, slave) for the FEB/D and ASIC ID. Other function that can be useful for the user, allowing to power off/on the Front End Modules is:

```
./set_fem_power --power off  
./set_fem_power --power on
```

With the argument “off”, this program will power off all the FEMs in the system, including the Bias voltage for the SIPMs, thus allowing to safely remove or plug any detector or additional FEM, or even just turn on the lights safely.

This function can be applied with a corresponding switch in the “Power” section of the GUI, as well as a similar function that only turns “on” and “off” the bias voltage to the SIPMs and not the full FEM power.

4.5 Using an external trigger

It is possible for the TOFPET2 Evaluation Kit (FEB/D version, after February 2017) to accept signals for triggering. More information on this feature and how to set it in Software can be found in the document “Application note: Synchronization with external system.pdf”.

Chapter 5

Data processing

It was mentioned previously that the raw data written to disk with the data acquisition scripts does not have a correct detection time in seconds. The raw data contains only the measured values from the ASIC TDCs in ADC units. The conversion from these values into time does not follow a universal function (although it is very close to a linear relation) and requires a calibration function for each ASIC channel (see sec. 3.1.2). Also, if the acquisition was performed in “QDC” mode, a post-processing correction of the raw data delivered by the ASIC is also needed (see sec. 3.1.3). In order to use the calibration files and convert the raw data into data with time and energy information, one needs to use the processing applications provided by PETsys Electronics.

5.1 Singles application

The main processing application is “convert_raw_to_singles”. By default, this program provides a list of all registered events using minimal filtering (only excludes events where the measured time in ADCs is outside of the operational nominal range), writing by default, as output, a text file where each line corresponds to one event and each column to the following variables:

- Time of detection in picoseconds
- Energy of the pulse (in QDC mode, in arbitrary units of charge; in TOT mode, in nanoseconds).
- Absolute channel ID (see appendix A)

If the energy calibration file (see section 3.1.4) exists and is referenced in the reference configuration file, the energy value will also have arbitrary units of charge. In order to process the data (following the example from chapter 4), just type:

```
./convert_raw_to_singles --config data_folder/config.ini  
-i data_folder/data_prefix -o data_folder/data_singles.txt
```

In the above example, “data_folder/data_prefix” is the input (flag “-i”) raw data file prefix and “data_folder/data_singles.txt” is the output (flag “-o”) file.

As an example of the output of this application, “data_singles.txt” in this example (in TOT mode):

```
...  
746540587230 1.391052 114  
746540590891 1.612877 66  
746540587241 4.427330 72  
746540584157 14.772469 68  
746540584648 6.921204 70  
746540589927 1.254074 123  
746540588194 2.033409 74  
746540585346 4.360729 67
```

```

746540593966 1.428196 77
746540679252 17.219238 280
746540682757 1.993874 275
746540683588 4.370556 281
746540680081 13.847786 119
...

```

When the amount of data is very large, the text output file requires a large disk storage space. The application has the option to write the output file in a more compact binary format using the “--writeBinary” flag. If this option is set, the same three variables per event will be written to the output file in a format of 64 bit integer (time in picoseconds), 32 bit floating point (energy or ToT, depending on the acquisition mode) and 32 bit integer (channel ID). The C code structure for each event is:

```

1 struct Event {
2     long long time;
3     float e;
4     int id;
5 };

```

If using the binary output option, the output file defined after the “-o” flag is indeed a prefix, since two files will be created. In this way, the user should not define an extension for the file, and can run the application as:

```

./convert_raw_to_singles --config data_folder/config.ini
-i data_folder/data_prefix -o data_folder/data_singles --writeBinary

```

Two files will be created in “data_folder”: a “data_singles.lidat” containing the binary data with the structure presented above and a “data_singles.lidx” which contains information of the event range corresponding to different parameters acquired during a parameter scan as described in section 4.2.1. If the data acquisition did not involve a parameter scan, then this file will not be relevant but will still be created (containing a single line).

A third output data type is possible with the application “convert_raw_to_singles”, consisting of a ROOT file with the data in the form of a ROOT TTree named “data” with one event per entry. This TTree will contain some more information useful besides the three entries described above. The full TTree structure is:

- “step1” and “step2” are the values of step1 and step2 from the acquisition with multiple parameters. If no scan is performed, both this variables are set to 0.
- “time” is the time of the event, in picoseconds.
- “channelID” is the absolute channel ID of the event (see appendix A).
- “tac” is the TAC number for this event, ranging from 0 to 3.
- “tot” is the Time-over-Threshold for the event, in nanoseconds. In QDC acquisition mode, this time corresponds to the time of integration, while in TOT mode it is the difference between the time stamps at the E and t1 discriminators.
- “energy” is the energy of the event (in QDC mode, in ADC units; in TOT mode, in nanoseconds).
- “channelIdleTime” is the time between the event and the previous event in the same channel.
- “tacIdleTime” is the time between the event and the previous event in the same TAC.
- “xi” and “yi” are the index position in the X and Y directions of the pixel connected to channelID (obtained from the channel map file described in Sec. 3.2.1).
- “x”, “y” and “z” are the physical coordinates of the pixel connected to channelID (obtained from the channel map file described in Sec. 3.2.1).
- “tqT” is the fine time (time for crossing the time threshold) measured directly by the TDC in units of clock periods.

- “tqE” is the fine time (time for crossing the energy threshold) measured directly by the TDC in units of clock periods.

For the ROOT output, use the option “--writeRoot”, so the application should be run as:

```
./convert_raw_to_singles --config data_folder/config.ini
-i data_folder/data_prefix -o data_folder/data_singles.root --writeRoot
```

Option “--help” prints to screen all the arguments and flags for “convert_raw_to_singles”.

5.2 Coincidence application

With the single event data, the user can create data analysis routines and extract any required information. Nevertheless, PETsys Electronics provides an application (“convert_raw_to_coincidence”) designed to filter and group the events based on geometrical and timing criteria, identifying and writing only coincidence event data, useful for PET related applications. The application will group single events that occur in a given geometrical region (only relevant if using SIPM arrays), and identify any two of these events as being in coincidence if they are detected with a time difference smaller than 10 ns (by default) and occur in different trigger regions. The trigger regions are defined in the channel map described previously and selected through a trigger table map (Sec. 3.2).

In order to use the coincidence application, just type:

```
./convert_raw_to_coincidence --config data_folder/config.ini
-i data_folder/data_prefix -o data_folder/data_coincidences.txt
```

The default output file will be also a text file. Each line corresponds to a coincidence event. As described above, one coincidence event consists of two single events, occurring in different trigger regions. Besides the information delivered by the singles application, two more integer numbers are provided for each of the two events in coincidence. The ten columns correspond to the following variables:

- “mh_n1”, the number of events that belong to the same group of this particular event (for the event in the trigger region of higher ID number). If “mh_n” is 1, then no other events were detected within a radius of 100mm and a 100 ns time window (default grouping spacial and time windows).
- “mh_j1”, the number ID (from 0 to “mh_n”-1) of this event in the group it belongs to (ordered from higher to lower energy, for the event in the trigger region of higher ID number). For example, an event with “mh_j” set to 1 means this event has the second highest energy in the group it belongs to.
- Time of detection in picoseconds (for the event in the trigger region of higher ID number).
- Energy of the pulse, for the event in the trigger region of higher ID number (in QDC mode, in arbitrary units of charge; in TOT mode, in nanoseconds). If the energy calibration file (see section 3.1.4) exists and is referenced in the reference configuration file, the energy value will also have arbitrary units of charge.
- Absolute channel ID, for the event in the trigger region of higher ID number (see appendix A).
- “mh_n2”, same as “mh_n1”, but for the coincidence event in the trigger region of lower ID number.
- “mh_j2”, same as “mh_n2”, but for the coincidence event in the trigger region of lower ID number.
- Time of detection in picoseconds (for the event in the trigger region of lower ID number).
- Energy of the pulse, for the event in the trigger region of lower ID number (in QDC mode, in ADC units; in TOT mode, in nanoseconds).
- Absolute channel ID, for the event in the trigger region of lower ID number (see appendix A).

An example of the output data stored in “data_folder/data_coincidences.txt” (by default, writing only the more energetic event in each group - mh-j=0):

```
...
16 0 746540679252 17.219238 280 2 0 746540688390 2.190071 115
21 0 746542092613 13.789963 289 7 0 746542093405 14.952671 115
14 0 746543556436 18.033173 278 17 0 746543557528 11.129692 122
18 0 746559438500 16.535572 283 27 0 746559437824 20.985302 87
7 0 746582738153 8.471455 270 8 0 746582738245 10.569595 101
3 0 746639786672 3.113194 260 21 0 746639784992 15.615772 109
16 0 746653183836 17.341839 278 21 0 746653184244 19.260880 119
6 0 746743348031 6.681160 302 13 0 746743346465 17.183113 97
2 0 746752823535 1.654106 270 14 0 746752821572 10.654236 98
15 0 746785502951 15.923786 279 10 0 746785503584 14.865990 122
31 0 746791064825 23.305775 267 9 0 746791065377 8.829895 73
21 0 746793932627 16.414318 263 16 0 746793933032 13.508621 99
19 0 746913280769 18.704937 271 7 0 746913281535 6.642433 110
10 0 746921420200 17.198643 275 14 0 746921421380 12.888512 125
9 0 746922094190 17.683727 275 14 0 746922095552 9.447754 123
24 0 746930656137 19.851555 269 8 0 746930657504 5.457588 107
14 0 746976127033 11.758198 283 2 0 746976128004 3.096554 106
10 0 747018393456 8.486469 279 14 0 747018393842 12.628952 126
2 0 747023540299 3.772789 274 5 0 747023541428 3.700508 123
18 0 747036209898 16.228996 285 1 0 747036212717 3.591759 127
15 0 747036694818 18.114315 275 5 0 747036697098 4.196793 121
26 0 747135818329 12.057774 288 33 0 747135817700 19.561619 65
20 0 747165260554 11.689510 258 14 0 747165260386 14.569408 110
8 0 747186019605 16.644855 273 33 0 747186019647 21.217892 100
4 0 747203593353 7.317780 274 13 0 747203592466 24.086327 78
...
```

Again, using the option “--writeBinary”, the output file will be a compact binary with 10 variables per event and the data format will be: 64 bit integer (time in picoseconds), 32 bit floating point (ToT or Energy) and 32 bit integer (channel ID), 64 bit integer (time in picoseconds), 32 bit floating point (ToT or Energy) and 32 bit integer (channel ID). The full C code structure for each event is:

```
1 struct Event {
2     uint8_t mh_n1;
3     uint8_t mh_j1;
4     long long time1;
5     float e1;
6     int id1;
7     uint8_t mh_n2;
8     uint8_t mh_j2;
9     long long time2;
10    float e2;
11    int id2;
12 };
```

Also, a third output data type is possible with the application “convert_raw_to_coincidence”, consisting of a ROOT file with the data in the form of a ROOT TTree named “data” with one coincidence event per entry. There will be two sets of “time”, “channelID”, “tac”, “tot”, “energy”, “mh_n”, “mh_j”, “xi”, “yi”, “x”, “y”, “z”, one for each event in a coincidence, as branches of the TTree “data”.

Several other command line options are available to control the filtering of coincidence events during processing and when writing to output file. Option “--help” prints to screen all the arguments and flags for “convert_raw_to_coincidence”.

5.3 Group application

Other application provided by PETsys Electronics that can be useful for processing and analysing raw data is “convert_raw_to_group”. This program is very similar to the singles application, in the sense that processes and retrieves time and energy information for all events applying minimal filtering, the difference being that it identifies groups of events (originating from a single Gamma photon interacting in a crystal block or matrix when coupled to a SIPM array, for example), based on their time of arrival and detection position. The program will consider two (or more) events to part of the same group if they are detected (by default) within a 100ns time window and less than 100 mm distance from another in the same trigger region (these values are configurable). In order to use the group application, type:

```
./convert_raw_to_group --config data_folder/config.ini -i data_folder/data_prefix  
-o data_folder/data_group.txt --writeMultipleHits 64
```

The option “--writeMultipleHits” defines the maximum number of events per group that will be written to the output file, in this example 64. The default output file will be in text format. Each line corresponds to one event. As described above, beside the same the information delivered by the singles application, two more integer numbers are provided. The 5 columns correspond to:

- “mh_n”, the number of events that belong to the same group of this particular event. If “mh_n” is 1, then no other events arrived within 100 mm and within a 100 ns time window.
- “mh_j”, the number ID (from 0 to “mh_n”-1) of this event in the group it belongs to (ordered from higher to lower energy). For example, an event with “mh_j” set to 1 means this event has the second highest energy in the group it belongs to. By default this number is always 0, unless the option “--writeMultipleHits” is defined.
- Time of detection in picoseconds (for the event in the trigger region of higher ID number).
- Energy of the pulse, for the event in the trigger region of higher ID number (in QDC mode, in arbitrary units of charge ; in TOT mode, in nanoseconds). If the energy calibration file (see section 3.1.4) exists and is referenced in the reference configuration file, the energy value will also have arbitrary units of charge.
- Absolute channel ID, for the event in the trigger region of higher ID number (see appendix A).

An example of the output data stored in “data_folder/data_group.txt”:

```
...  
13 0 746544972280 12.573620 94  
13 1 746544974401 3.998756 90  
13 2 746544976888 2.966228 93  
13 3 746544974937 2.549541 100  
13 4 746544975858 2.487907 95  
13 5 746544990825 1.420338 98  
13 6 746544980917 1.161053 88  
13 7 746544985430 0.659710 92  
13 8 746544974718 0.571350 75  
13 9 746544979109 0.508190 103  
13 10 746544983920 0.259937 96  
13 11 746544982585 0.181274 97  
13 12 746545017441 0.137691 105  
7 0 746546240112 8.429253 79  
7 1 746546242364 2.906200 66  
7 2 746546242511 2.823254 72  
7 3 746546243766 2.694466 70  
7 4 746546245394 1.887131 68  
7 5 746546243114 1.364708 123  
7 6 746546276007 0.162373 114
```

```

1 0 746547881873 1.297798 310
8 0 746555808476 9.199627 275
8 1 746555812228 1.431965 273
8 2 746555838748 1.253864 279
8 3 746555813751 1.213791 281
8 4 746555818542 1.046593 278
8 5 746555838681 0.488628 280
8 6 746555813239 0.272110 272
8 7 746555825509 0.176234 284
...

```

Again, when using the option “--writeBinary”, the output file will be in compact binary format, with 5 variables per event and the data format will be: 64 bit integer (time in picoseconds), 32 bit floating point (ToT or Energy) and 32 bit integer (channel ID), 64 bit integer (time in picoseconds), 32 bit floating point (ToT or Energy) and 32 bit integer (channel ID). The full C code structure for each event is:

```

1 struct Event {
2     uint8_t mh_n;
3     uint8_t mh_j;
4     long long time;
5     float e;
6     int id;
7 };

```

Also, a third output data type is possible with the application “convert_raw_to_group”, consisting of a ROOT format file, containing the data in the form of a ROOT TTree named “data” with one event per entry. There will be one set of “time”, “channelID”, “tac”, “tot”, “energy”, “mh_n”, “mh_j”, “xi”, “yi”, “x”, “y”, “z”, as branches of the TTree “data”.

5.4 Raw to raw application

It was mentioned above that the raw data (time and energy) originated from the ASIC and FPGA needs to be corrected with calibration functions, hence the need for processing applications. Nevertheless, as a debugging tool to check if there is some problem in the raw data, the user may want to check this data before the application of any calibration. To do so, and since the raw data is in binary format, PETSYS Electronics also provides an application that converts the raw data into a ROOT TTree format.

In order to use it, just type:

```

./convert_raw_to_raw --config data_folder/config.ini
-i data_folder/data_prefix -o data_folder/data_raw.root

```

The full TTree structure of the output data is:

- “step1” and “step2” are the values of step1 and step2 from the acquisition with multiple parameters. If no scan is performed, both this variables are set to 0.
- “channelID” is the absolute channel ID of the event (see appendix A).
- “tacID” is the TAC number for this event, ranging from 0 to 3.
- “tcoarse” is the coarse time stamp measured at the t1 discriminator, in clock periods, from 0 to 1023.
- “tfine” is the fine time stamp measured by the TDC at the t1 discriminator, in ADC units, typically ranging from 200 to about 400.
- “ecoarse”, in TOT acquisition mode, is the coarse time stamp measured at the E discriminator, in clock periods, from 0 to 1023; in QDC acquisition mode, is the coarse time stamp corresponding to the charge integration ending, in clock periods, from 0 to 1023.

- “efine” in TOT acquisition mode, is the fine time stamp measured by the TDC at the E discriminator, in ADC units, typically ranging from 200 to about 400; in QDC acquisition mode, is the charge integration value, in ADC values.
- “frameID” identifies the frame of data to each the event belongs. Each frame contains 1024 clock periods.

5.5 Data Processing with GUI

The above sections described the available processing applications and output options. All these functionalities and options are available in the GUI, where the user can choose the raw file name (or prefix) to be processed, the output file name, the event type of the processed file (“Singles”, “Coincidences”, “Groups” or “Raw”) and the output data format: “Binary”, “Text” or “ROOT”. The user can also choose two other options: “Fraction”, defining the fraction of the raw data that will be effectively processed (from 0 to 100%) and the “Number of Hits”, which is only applicable when processing to Coincidences or Grouped events, as described above.

Appendix A

Absolute ID of Channels, ASICs and HV DAC channels

The software and firmware assign an absolute ID to each ASIC, channel, and HV DAC channel. This ID numbering is related directly to the FEB/D port where a given TOFPET2 ASIC Test board is plugged.

The absolute IDs are constructed in the following way:

$$\text{Absolute Channel ID} = 64 \times \text{chipID} + \text{channelID}$$

where:

- **chipID:** The ASIC ID, related to the position where it is connected in the FEB/D. This number can range from 0 to 15, where 0 and 1 correspond to the ASICs belonging to the Front End Module connected in port F1 of the FEB/D, 2 and 3 to the ones connected in port F2 and so on up to port F8. In a given Front End Module, ASIC 0 is the one belonging to the FEB/A connected to J1 of the FEB/I and ASIC 1 is the one belonging to the FEB/A connected to J2 of the FEB/I.
- **channelID:** The channel ID inside the ASIC, from 0 to 63. In order to know exactly what input is connected to a given ASIC channel, please refer to tables 2 and 3 of the “TOFPET2 ASIC SIPM Readout System - Hardware user Guide”.

Regarding the Bias DAC channel identification, used in the Bias calibration file and Bias settings file, it follows the same logic for identification of the FEB/D, using “portID” and “slaveID” and a channelID that is related to the specific HV DAC channel within the FEB/D. In order to know where each channelID connects to, the user may refer to the tables present in the folder “TOFPET2 > Mapping tables” on the Client Package Folder.